

Learning Objectives:

1. What are activation functions?
2. Types of activation functions in neural networks
3. Most widely used activation functions
4. How to choose an activation function for our analysis?
5. Coding activation functions in python

What are activation functions?

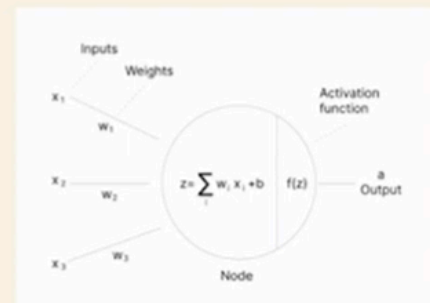
Activation functions are a critical part of the design of a neural network.

An **activation function** in a neural network defines how the weighted sum of the input is transformed into an output from a node or nodes in a layer of the network.

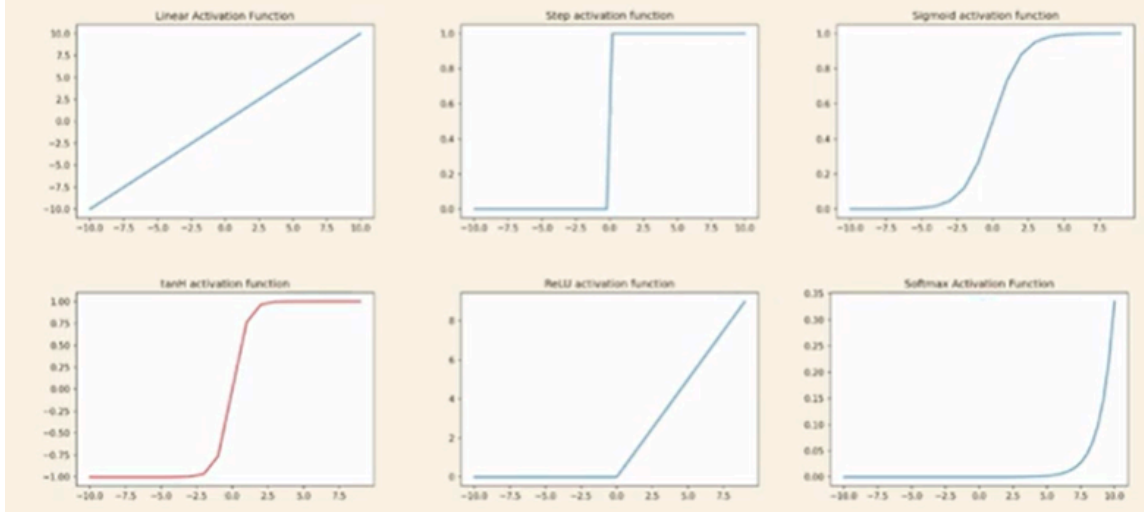
Sometimes the activation function is called a **“transfer function”**.

If the output range of the activation function is limited, then it may be called a **“squashing function”**.

Many activation functions are nonlinear and may be referred to as the **“nonlinearity”** in the layer or the network design.



Types of Activation functions



Why Activation functions?

- Tonnes of data is available
 - Data is non-structured
 - No defined line between useful and non-useful data
 - Noisy data will not give required output

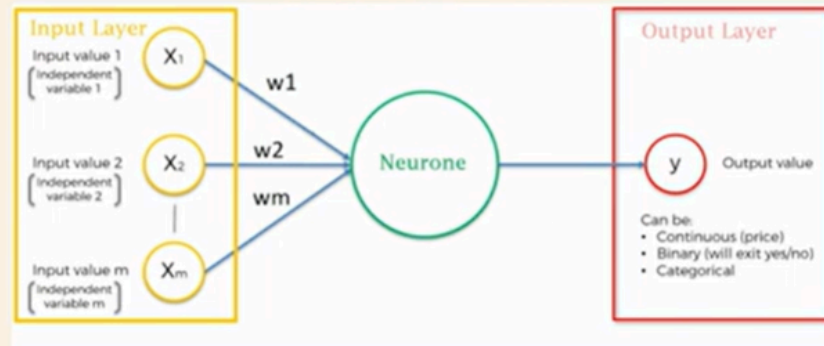
Activation functions help us to deal with it.

Activation functions are mathematical equations that determine the output of a neural network model. They help the network to use the important information and suppress the noise.

- In a neural network, activation functions are utilized to **bring non-linearities** into the decision border.
- The goal of introducing nonlinearities in data is to **simulate a real-world situation**.

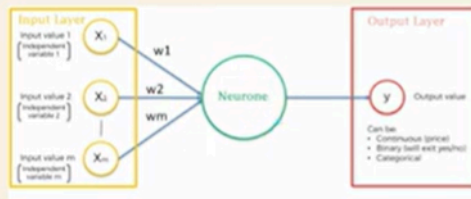
Almost much of the data we deal with in real life is nonlinear. This is what makes neural networks so effective.

Where exactly Activation functions are used in neural network?



A simple model of neuron commonly known as the **perceptron**. It can be divided into 3 parts: **Input layer, neuron, and output layer.**

Where exactly Activation functions are used in neural network?



We have input as $X_1, X_2, X_3, \dots, X_m$, and the weights associated with those inputs as $w_1, w_2, w_3, \dots, w_m$.

We first calculate the weighted sum of the input i.e.

$$\sum_{i=1}^m w_i * x_i$$

Then Activation function is applied as,

$$\phi\left(\sum_{i=1}^m w_i * x_i\right)$$

The calculated value is then passed to the next layer if present.

Where exactly Activation functions are used in neural network?

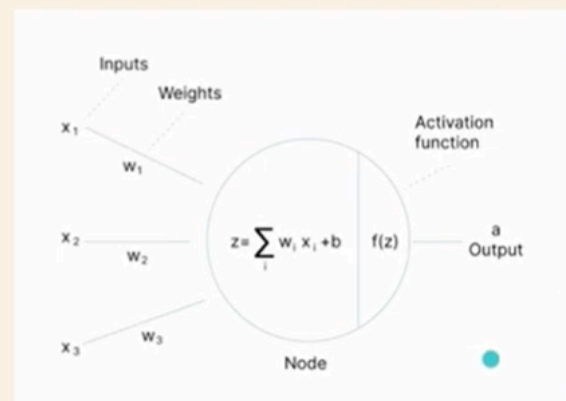
We first calculate the weighted sum of the input i.e.

$$\sum_{i=1}^m w_i * x_i$$

Then Activation function is applied as,

$$\phi\left(\sum_{i=1}^m w_i * x_i\right)$$

The calculated value is then passed to the next layer if present.



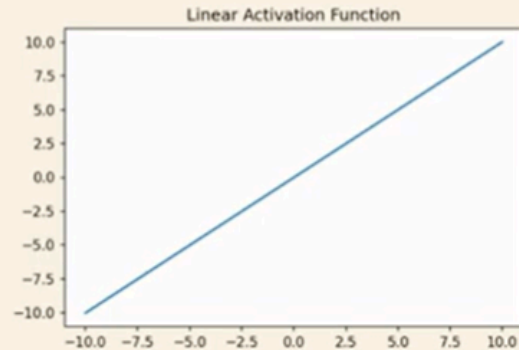
Linear Activation function

The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the **activation is proportional to the input**.

The function **doesn't do anything to the weighted sum of the input**, it simply spits out the value it was given as shown previously.

Linear

$$f(x) = x$$

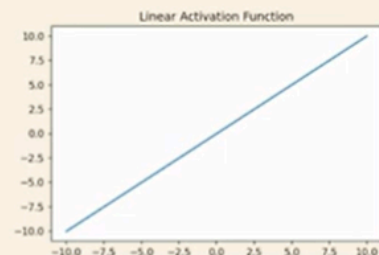


So, essentially, a linear activation function turns the neural network into just one layer

Linear Activation function

A linear activation function has two major problems :

1. It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x .
2. All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer.



So, essentially, a linear activation function turns the neural network into just one layer.

Non-linear Activation functions

The limited power **linear activation function** (simply a linear regression model), this does not allow the model to create complex mappings between the network's inputs and outputs.

Non-linear activation functions solve the following limitations of linear activation functions:

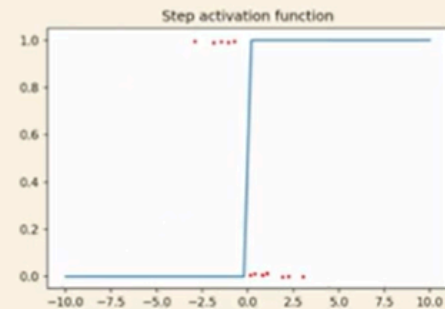
- They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

Binary Step Function

Binary step function is a threshold-based activation function which means:

- after a certain threshold neuron is activated
- below the said threshold neuron is deactivated.

In the graph, the threshold is zero. This activation function can be used in binary classifications as the name suggests, however it can not be used in a situation where you have multiple classes to deal with.



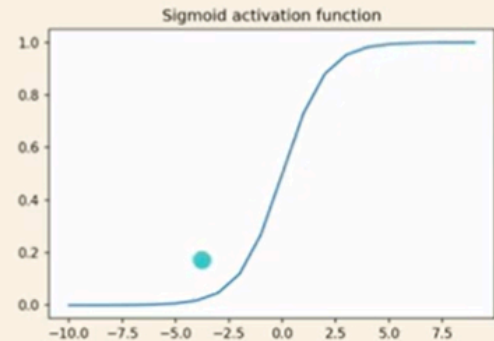
Sigmoid/Logistic Function

- Smooth curve between 0 and 1
- This function takes any real value as input and outputs values in the range of 0 to 1.
- The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0, as shown below.

Sigmoid / Logistic

$$f(x) = \frac{1}{1 + e^{-x}}$$

Where e is a mathematical constant, which is the base of the natural logarithm.



Sigmoid/Logistic Function

Advantages:

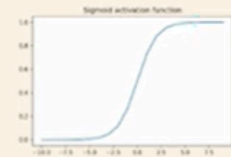
Why sigmoid function is most widely used:

- It is commonly used for models where we have **to predict the probability** as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- It normalizes the output of each neuron.
- The function is **differentiable and provides a smooth gradient**, i.e., preventing jumps in output value. This is represented by an **S-shape** of the sigmoid activation function.

However, Sigmoid function makes almost no change in the prediction for very high or very low inputs which ultimately results in neural network refusing to learn further, this problem is known as the vanishing gradient.

Disadvantages:

- Computationally expensive
- Outputs not zero centered
- Vanishing gradient—for very high or very low values of X, there is almost no change to the prediction, causing a **vanishing gradient problem**. This can result in the network refusing to learn further, or being too slow to reach an accurate prediction.

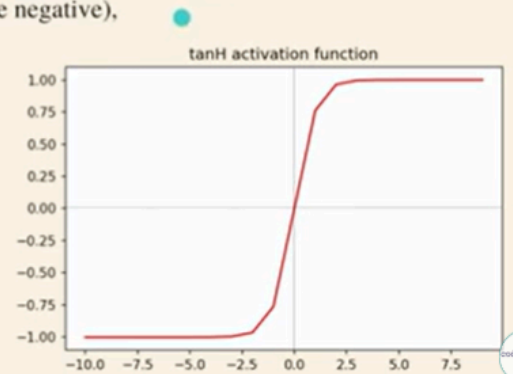


Tanh Function (Hyperbolic Tangent)

- Very similar to the sigmoid/logistic activation function
- Even has the same S-shape
- But the difference in output range of **-1 to 1**.
- In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.

$$\text{Tanh}$$
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

Where **e** is a mathematical constant, which is the base of the natural logarithm.

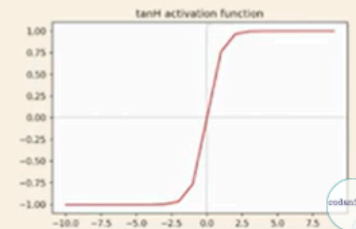


Tanh Function (Hyperbolic Tangent)

Advantages:

- Zero Centered
- The prediction is simple, ie based on a threshold probability value.

However, tanh also comes with the vanishing gradient problem just like sigmoid function.



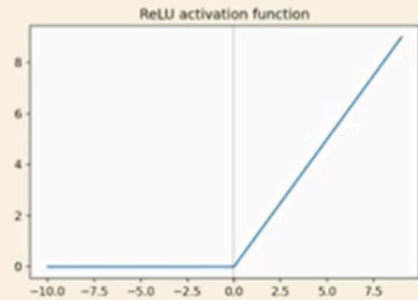
ReLU (Rectified Linear Unit) Function

Advantages:

- Since only a certain number of neurons are activated, the ReLU function is far more **computationally efficient** when compared to the sigmoid and tanh functions.
- ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

Disadvantages:

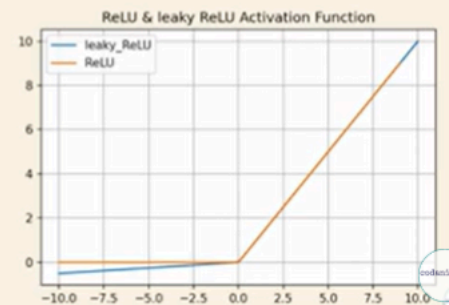
- When the input is zero or a negative value, the function outputs zero and it hinders with the back-propagation. This problem is known as the dying ReLU problem.



Leaky ReLU (Rectified Linear Unit) Function

- Leaky ReLU is the most common and effective method to solve a dying ReLU problem.
- It is nothing but an improved version of the ReLU function.
- It adds a slight slope in the negative range to prevent the dying ReLU issue.

$$\begin{array}{ccc} \text{ReLU} & \longrightarrow & \text{Leaky ReLU} \\ f(x) = \max(0, x) & & f(x) = \max(0.1x, x) \end{array}$$



Parametric ReLU (Rectified Linear Unit) Function

Advantages:

- Parametric ReLU is another variant of ReLU that aims to solve the problem of gradient's becoming zero for the left half of the axis.
- This function provides the slope of the negative part of the function as an argument a . By performing backpropagation, the most appropriate value of a is learnt.

Parametric ReLU

$$f(x) = \max(ax, x)$$

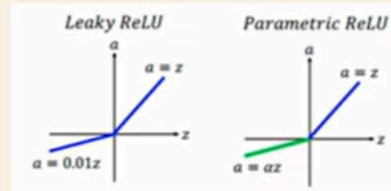
Where "a" is the slope parameter for negative values.

Parametric ReLU (Rectified Linear Unit) Function

Advantages:

- Parametric ReLU is another variant of ReLU that aims to solve the problem of gradient's becoming zero for the left half of the axis.
- This function provides the slope of the negative part of the function as an argument a . By performing backpropagation, the most appropriate value of a is learnt.

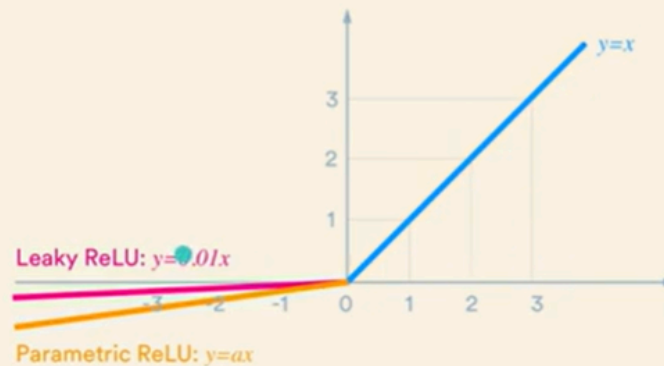
$$\begin{array}{ccccc} \text{ReLU} & & \text{Leaky ReLU} & & \text{Parametric ReLU} \\ f(x) = \max(0, x) & \longrightarrow & f(x) = \max(0.1x, x) & \longrightarrow & f(x) = \max(ax, x) \end{array}$$



Where "a" is the slope parameter for negative values.

Parametric ReLU (Rectified Linear Unit) Function

$$\begin{array}{ccccc} \text{ReLU} & & \text{Leaky ReLU} & & \text{Parametric ReLU} \\ f(x) = \max(0, x) & \longrightarrow & f(x) = \max(0.1x, x) & \longrightarrow & f(x) = \max(ax, x) \end{array}$$



Parametric ReLU (Rectified Linear Unit) Function

Advantages:

- The parametric ReLU function is used when the leaky ReLU function still fails at solving the problem of dead neurons, and the relevant information is not successfully passed to the next layer.

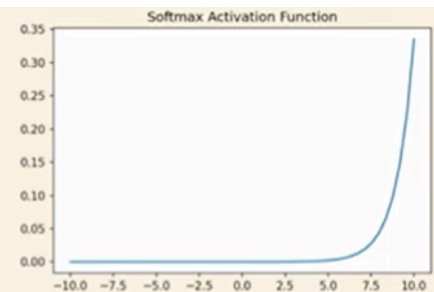
Disadvantages:

- This function's limitation is that it may perform differently for different problems depending upon the value of slope parameter a .

Softmax activation Function

- The softmax activation function is the generalized form of the sigmoid function for multiple dimensions.
- It is the mathematical function that converts the vector of numbers into the vector of the probabilities.
- The softmax activation function is commonly used as an **activation function in the case of multi-class classification** problems in machine learning.
- The output of the softmax is interpreted as the probability of getting each class.

The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.



Softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Softmax activation Function

The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.

But—

This function faces certain problems.

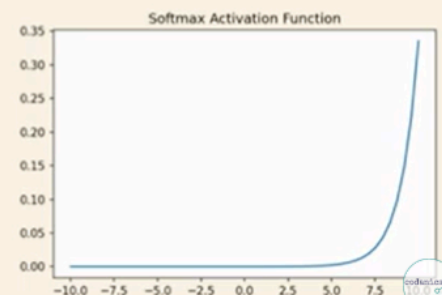
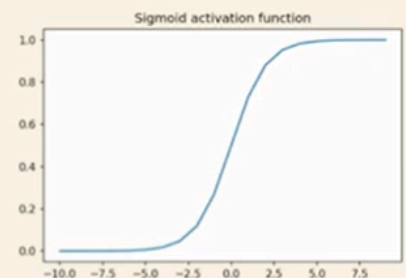
e.g:

Let's suppose we have five output values of 0.8, 0.9, 0.7, 0.8, and 0.6, respectively.

How can we move forward with it?

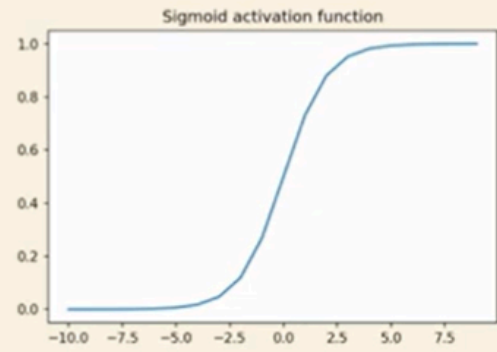
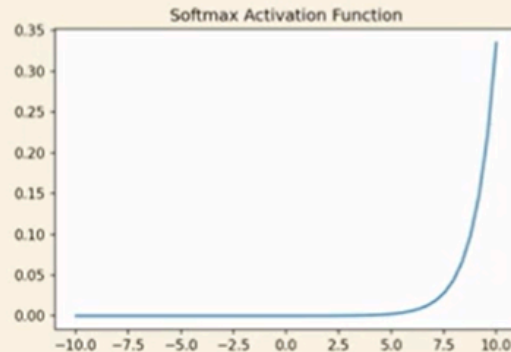
The answer is: We can't.

The above values don't make sense as the sum of all the classes/output probabilities should be equal to 1.



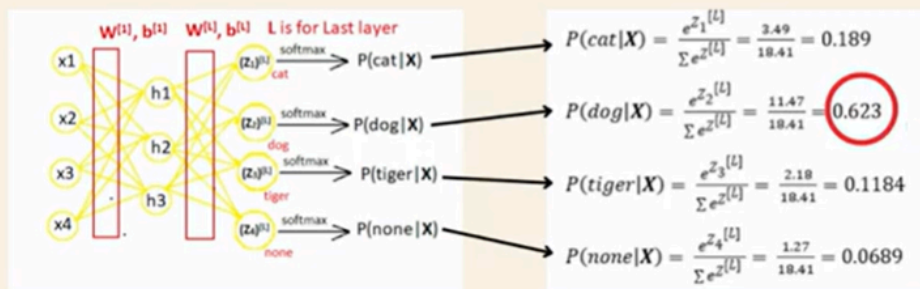
Softmax activation Function

- Softmax function is described as a combination of multiple sigmoids.
- It calculates the relative probabilities. Similar to the sigmoid/logistic activation function, the SoftMax function returns the probability of each class.
- It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification.



Softmax activation Function, Example:

Let's learn about the working of the softmax function with an example. Let's consider a neural network that classifies a given image, whether it is of cat, dog, tiger, or none. Let X is the feature vector (i.e. $X = \{x_1, x_2, x_3, x_4\}$).



By observing the probability distribution, we can say that the supplied image is of a dog.

Softmax activation Function

Advantages:

- It can be used for multiclass classification
- It normalizes the outputs for each class between 0 and 1, and divides by their sum, giving the probability of the input value being in a specific class.
- For neural networks that need to categorize inputs into numerous categories, Softmax is often employed **exclusively for the output layer**.

How to choose an activation function

General Rules to use activation functions:

Use **sigmoid** function in output layer, use **tanh** in all other places.
But both of these have vanishing gradient problem.

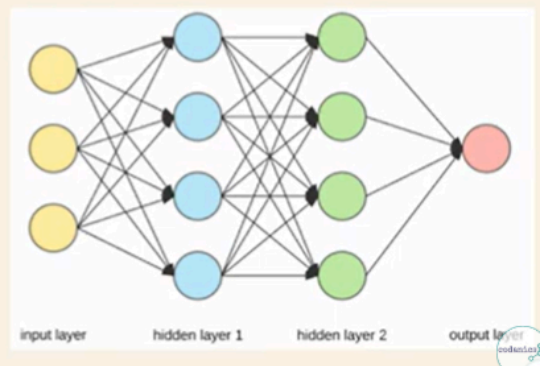
- Therefore, **ReLU** can be used in case of both above.
- **ReLU** is most popularly used function for hidden layers.
- Computationally effective math due to simplicity.
- However, it also has vanishing gradient problem.

Leaky ReLU or **Parametric ReLU** can be used here then to reduce the computation.

Activation functions for Hidden layers:

A **hidden layer** in a neural network is a layer that **receives input** from another layer (such as another hidden layer or an input layer) and **provides output** to another layer (such as another hidden layer or an output layer).

- A **hidden layer** does not directly contact input data or produce outputs for a model, at least in general.
- A neural network may have zero or more hidden layers.
- Typically, a **differentiable nonlinear activation function** is used in the hidden layers of a neural network. This allows the **model to learn more complex** functions than a network trained using a **linear activation function**.



Activation functions for Hidden layers:

Most widely used activation functions for hidden layers:

- Rectified Linear Activation (**ReLU**)
 - Logistic (**Sigmoid**)
 - Hyperbolic Tangent (**Tanh**)
-
- ReLU is the most common activation function used for the hidden layers.
 - Simple to implement and effective at overcoming the limitations Sigmoid and Tanh.
 - Less susceptible to vanishing gradients that prevent deep models from being trained.
 - Although it can suffer from other problems like **saturated** or “**dead**” units.
 - So we can use **Leaky ReLU** or **Parametric ReLU**.

Activation functions for Hidden layers:

How to choose the best activation function?

- A neural network will almost always have **the same activation function in all hidden layers**.
- Traditionally, the **sigmoid activation function** was the default activation function in the 1990s.
- Perhaps through the mid to late **1990s to 2010s**, the **Tanh function** was the default activation function for hidden layers.

"... the hyperbolic tangent activation function typically performs better than the logistic sigmoid."

(Page 195, Deep Learning, 2016.)

Both the **sigmoid and Tanh** functions can make the model more susceptible to problems during training, via the so-called **vanishing gradients** problem.

Activation functions for Hidden layers:

How to choose the best activation function?

- Modern neural network models with common architectures, such as MLP and CNN, will make use of the ReLU activation function, or extensions.
- **Recurrent networks** still commonly use **Tanh or sigmoid** activation functions, or even both.
 - For example, the LSTM (Long Short Term Memory) commonly uses the **Sigmoid activation** for recurrent connections and the **Tanh activation** for output.

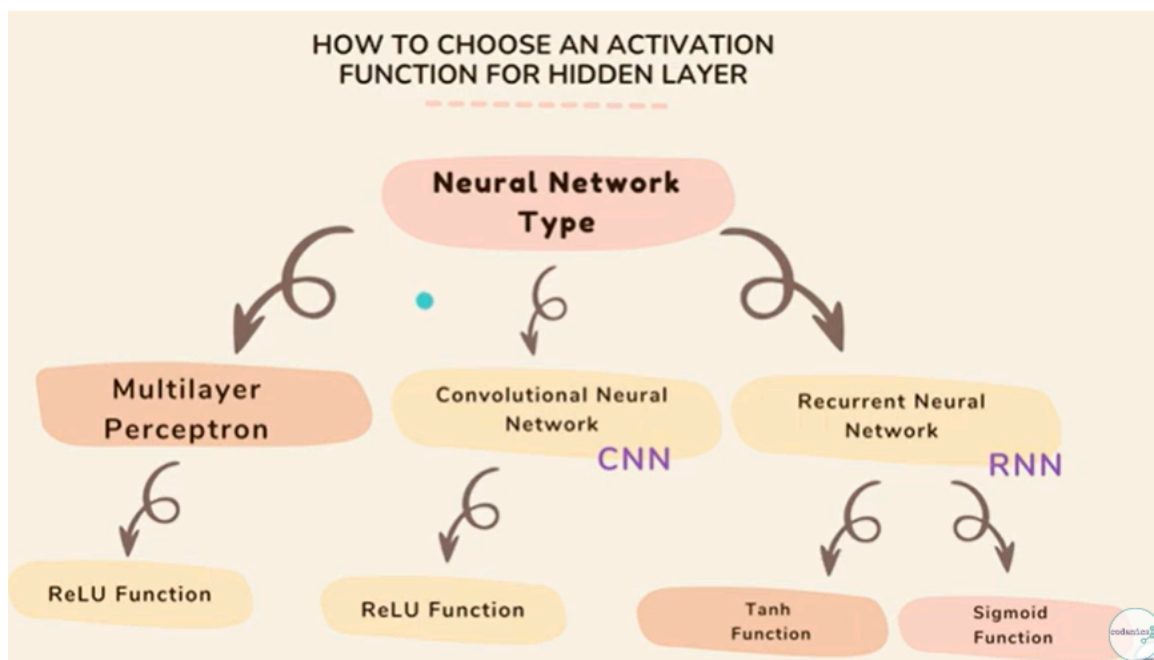
Activation functions for Hidden layers:

How to choose the best activation function?

The activation function used in hidden layers is typically chosen based on the type of neural network architecture.

- **Multilayer Perceptron (MLP):** ReLU activation function.
- **Convolutional Neural Network (CNN):** ReLU activation function.
- **Recurrent Neural Network:** Tanh and/or Sigmoid activation function.

If you're unsure which activation function to use for your network, try a few and compare the results.



Activation functions for output layers:

- The output layer is the layer in a neural network model that directly outputs a prediction.
- All feed-forward neural network models have an output layer.

Most widely used activation functions for output layers:

- Linear Activation function
- Logistic (**Sigmoid**)
- Softmax

The linear activation function is also called “**identity**” (multiplied by 1.0) or “**no activation.**”

This is because the linear activation function does not change the weighted sum of the input in any way and instead returns the value directly.

Activation functions for output layers:

How to choose the best activation function?

- Based on the type of prediction problem that you are solving
 - Specifically, the type of variable that is being predicted.
- Divide prediction problems into two main groups:
 - Predicting a categorical variable (**Classification**)
 - Predicting a numerical variable (**Regression**)

If your problem is a regression problem, you should use a linear activation function.

- Regression: One node, linear activation.

If your problem is a classification problem, then there are three main types of classification problems and each may use a different activation function.

Activation functions for output layers:

How to choose the best activation function?

Predicting a probability is not a regression problem; it is classification.

In all cases of classification, your model will predict the probability of class membership (e.g. probability that an example belongs to each class) that you can convert to a crisp class label by

- rounding (**for sigmoid**)
- argmax (**for softmax**).

If there are two mutually exclusive classes (binary classification),

- then your output layer will have one node and a **sigmoid activation function** should be used.

If there are more than two mutually exclusive classes (multiclass classification),

- then your output layer will have one node per class and a **softmax activation** should be used.

If there are two or more mutually inclusive classes (multilabel classification),

- then your output layer will have one node for each class and a **sigmoid activation function** is used.

Activation functions for output layers:

How to choose the best activation function?

- **Binary Classification:** One node, sigmoid activation.
- **Multiclass Classification:** One node per class, softmax activation.
- **Multilabel Classification:** One node per class, sigmoid activation.

